

Формальные грамматики и языки

Теория автоматов и формальных языков

Лекция №4

Направления развития теории грамматик

- 1) Построение формальной (математической) лингвистики.
- 2) С появлением трансляторов проблема перевода во многом определила построение общей теории вычислительных машин, а сами языки программирования стали все более приближаться к формально построенным математическим конструкциям.

Направления развития теории грамматик

- 3) Построение формальных моделей динамических систем.
- 4) Общая теория алгоритмов как ветвь современной математики.

Теория формальных языков и грамматик

- Теория формальных языков и грамматик является разделом математической лингвистики - специфической математической дисциплины, ориентированной на изучение структуры естественных и искусственных языков.

Формальные языки

- На интуитивном уровне *язык* можно определить как некоторое множество предложений с заданной структурой, имеющих определенное значение.
 - Правила, определяющие допустимые конструкции языка, составляют *синтаксис языка*.
 - Значение, или смысл фразы, определяется *семантикой языка*.

Виды описания языков

- 1) *Порождающее*, которое предполагает наличие алгоритма, последовательно порождающего все правильные предложения языка;
- 2) *распознающее*, которое предполагает наличие алгоритма, определяющего принадлежность любой фразы данному языку.

Основные понятия порождающих грамматик

- **Алфавит** - это непустое конечное множество. Элементы алфавита называются символами.
- **Цепочка** над алфавитом $\Sigma = \{a_1, a_2, \dots, a_n\}$ есть конечная последовательность элементов a_i . Множество всех цепочек над алфавитом Σ обозначается Σ^* .
- **Длина цепочки** x равна числу ее элементов и обозначается $|x|$.
- Цепочка нулевой длины называется пустой и обозначается символом ϵ .

Множество цепочек алфавита

- Пусть имеется алфавит $\Sigma = \{a, b\}$.
- Тогда множество всех цепочек определяется следующим образом: $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, \dots\}$.

Цепочки алфавита

- Цепочки x и y равны, если они одинаковой длины и совпадают с точностью до порядка символов, из которых они состоят.
- Над цепочками x и y определена операция **сцепления** (конкатенации, произведения), которая получается дописыванием символов цепочки y за символами цепочки x .

Формальный язык

- Язык L над алфавитом Σ представляет собой множество цепочек над Σ .
- Необходимо различать пустой язык $L=\emptyset$ и язык, содержащий только пустую цепочку: $L=\{\epsilon\} \neq \emptyset$.
- Формальный язык L над алфавитом Σ - это язык, выделенный с помощью конечного множества некоторых формальных правил.

Операции над алфавитами

- Пусть M и L - языки над алфавитами. Тогда конкатенация $LM = \{xu \mid x \in L, u \in M\}$.
В частности, $\{\varepsilon\}L = L\{\varepsilon\} = L$.
- Используя понятие произведения, определим итерацию L^* и усеченную итерацию L^+ множества L .

$$L^* = \bigcup_{i=0}^{\infty} L^i,$$

$$L^+ = \bigcup_{i=1}^{\infty} L^i,$$

где i - степень языка,

L определяется рекурсивно следующим образом:

$$L^0 = \{\varepsilon\};$$

$$L^1 = L;$$

$$L^{n+1} = L^n L = L L^n ;$$

$$\{\varepsilon\}L = L\{\varepsilon\} = L.$$

Операции над алфавитами

- Например, если задан язык $L=\{a\}$, тогда
 $L^* = \{\epsilon, a, aa, aaa, \dots\}$,
 $L^+ = \{a, aa, aaa, \dots\}$.

Порождающая грамматика

Порождающая грамматика - это упорядоченная четверка $G = (V_T, V_N, P, S)$, где

- V_T - конечный алфавит, определяющий множество терминальных символов;
- V_N - конечный алфавит, определяющий множество нетерминальных символов;
- P - конечное множество правил вывода, т.е. множество пар следующего вида:
$$u \rightarrow v, \text{ где } u, v \in (V_T \cup V_N)^*;$$
- S - начальный символ (аксиома грамматики), $S \in V_N$.

Порождающая грамматика

- Из **терминальных символов** состоят цепочки языка, порожденного грамматикой.
- **Аксиомой** называется символ в левой части первого правила вывода грамматики.
- Для того чтобы различать **терминальные** и **нетерминальные** символы, принято обозначать терминальные символы **строчными**, а нетерминальные символы **заглавными** буквами латинского алфавита.

Порождающая грамматика

- В грамматике G цепочка x непосредственно порождает цепочку y , если $x = \alpha u \beta$, $y = \alpha v \beta$
- и $u \rightarrow v \in P$, т.е. цепочка y непосредственно выводится из x , что обозначается $x \Rightarrow y$.
- Языком, порождаемым грамматикой $G = (V_T, V_N, P, S)$, называется множество терминальных цепочек, выводимых в грамматике G из аксиомы:
- $L(G) = \{x \mid x \in V_T^*; S \Rightarrow^* x\}$, где $\Rightarrow^*$ - выводимость.

Пример порождающей грамматики

- Дана грамматика $G = (V_T, V_N, P, S)$, в которой $V_T = \{a, b\}$, $V_N = \{S\}$, $P = \{S \rightarrow aSb, S \rightarrow ab\}$.
- Определить язык, порождаемый этой грамматикой.

Пример порождающей грамматики

- Дана грамматика $G = (V_T, V_N, P, S)$, в которой $V_T = \{a, b\}$, $V_N = \{S\}$, $P = \{S \rightarrow aSb, S \rightarrow ab\}$.
- Решение. Используя рекурсию, выведем несколько цепочек языка, порождаемого данной грамматикой:
 $S \rightarrow ab$;
 $S \rightarrow aSb \Rightarrow aabb$;
 $S \rightarrow aSb \Rightarrow aaSbb \Rightarrow aaabbbb; \dots$
- Определим язык, порожденный данной грамматикой:
 $L(G) = \{a_n b_n \mid n > 0\}$.

Пример порождающей грамматики

- Предполагается, что правые части правил, для которых совпадают левые части, можно записать в одну строку с использованием разделителя.
- Так, правила вывода из приведенного примера можно записать следующим образом: $S \rightarrow aSb \mid ab$.

Классификация грамматик по Н. Хомскому

- Грамматики типа 0 - это грамматики, на правила вывода которых нет ограничений.
- Грамматики типа 1, или контекстные грамматики - это грамматики, все правила которых имеют следующий вид:
 - $xAy \rightarrow x\varphi y$, где $A \in V_N$,
 - $x, y, \varphi \in (V_N \cup V_T)^+$.

КС-грамматики

- Грамматики типа 2 - это бесконтекстные, или контекстно-свободные грамматики (КС-грамматики).

Правила вывода для этих грамматик имеют следующий вид: $A \rightarrow \varphi$, где $A \in V_N$, $\varphi \in (V_N \cup V_T)^*$.

Автоматные грамматики

- Грамматики типа 3 - это автоматные грамматики, которые делятся на два типа:
- а) левوليнейные (леворекурсивные), правила вывода для которых имеют следующий вид:

$$A \rightarrow Aa \mid a, \text{ где } A \in V_N ;$$

- б) праволинейные (праворекурсивные), правила вывода для которых имеют следующий вид:

$$A \rightarrow aA \mid a.$$

Определение типа формального языка

- Язык L называется языком типа i , если существует грамматика типа i , порождающая язык L .

Примеры грамматик и языков

- **Замечание:** если при описании грамматики указаны только правила вывода P , то будем считать, что большие латинские буквы обозначают нетерминальные символы, S - аксиома грамматики, все остальные символы - терминальные.

1) Язык типа 0: $L(G) = \{a^2b^{n^2-1} \mid n \geq 1\}$

$G: S \rightarrow aaCFD$

$F \rightarrow AFB \mid AB$

$AB \rightarrow bBA$

$CB \rightarrow C$

$Ab \rightarrow bA$

$AD \rightarrow D$

$Cb \rightarrow bC$

$bCD \rightarrow \varepsilon$

Примеры грамматик и языков

2) Язык типа 1: $L(G) = \{ a^n b^n c^n, n \geq 1 \}$

G: $S \rightarrow aSBC \mid abC$

$CB \rightarrow BC$

$bB \rightarrow bb$

$bC \rightarrow bc$

$cC \rightarrow cc$

3) Язык типа 2: $L(G) = \{ (ac)^n (cb)^n \mid n > 0 \}$

G: $S \rightarrow aQb \mid accb$

$Q \rightarrow cSc$

Примеры грамматик и языков

4) Язык типа 3: $L(G) = \{\omega \perp \mid \omega \in \{a,b\}^+, \text{ где нет двух рядом стоящих } a\}$

$G: S \rightarrow A\perp \mid B\perp$

$A \rightarrow a \mid Ba$

$B \rightarrow b \mid Bb \mid Ab$

Методика решения задач

- Пример 1.
- Дана порождающая грамматика $G = (V_T, V_N, P, S)$, в которой
$$V_T = \{a, d, e\},$$
$$V_N = \{B, C, S\},$$
$$P = \{S \rightarrow aB, B \rightarrow Cd \mid dC, C \rightarrow e\}.$$
- Выписать терминальные цепочки, порожденные данной грамматикой, и определить длину их вывода.

Методика решения задач. Пример 1

- $G = (V_T, V_N, P, S)$, в которой
 $V_T = \{a, d, e\}$,
 $V_N = \{B, C, S\}$,
 $P = \{S \rightarrow aB, B \rightarrow Cd \mid dC, C \rightarrow e\}$.
- Решение. Получим следующие терминальные цепочки:
 $S \rightarrow aB \rightarrow aCd \rightarrow aed$,
 $S \rightarrow aB \rightarrow adC \rightarrow ade$,
длина вывода которых равна 3.

Пример 2.

- Дана грамматика $G = (V_T, V_N, P, C)$,
в которой $V_T = \{a, b, c, d, e\}$,
 $V_N = \{A, B, C, D, E\}$,
 $P = \{A \rightarrow ed, B \rightarrow Ab, C \rightarrow Bc, C \rightarrow dD, D \rightarrow Ae, E \rightarrow bc\}$.
- Определить, принадлежит ли языку $L(G)$ цепочка $eadabcb$.

Пример 2.

Решение.

Выведем возможные терминальные цепочки из аксиомы с помощью заданных правил вывода:

$$C \rightarrow Bc \rightarrow Abc \rightarrow edbc,$$

$$C \rightarrow dD \rightarrow dAe \rightarrow dede.$$

- Так как первые же терминальные символы полученных цепочек не совпадают с заданной, можно сделать вывод о том, что цепочка eadabcb не принадлежит языку $L(G)$.

Лекция 5

- Грамматический разбор:
 - представление грамматики в виде графа;
- Преобразования КС-грамматик:
 - удаление правил вида $A \rightarrow B$;
 - построение неукорачивающей грамматики;
 - построение грамматики с продуктивными нетерминалами;
 - построение грамматики, аксиома которой зависит от всех нетерминалов;
 - удаление правил с терминальной правой частью;
 - построение эквивалентной праворекурсивной КС-грамматики.

Грамматический разбор

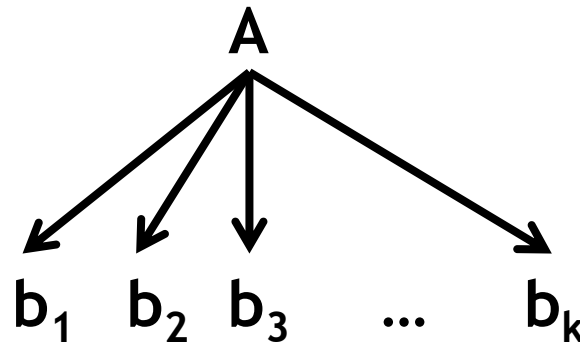
- В КС-грамматике может быть несколько выводов, эквивалентных в том смысле, что в них применяются одни и те же правила к одинаковым в промежуточном выводе цепочкам, но в различном порядке.

Грамматический разбор

- Например, для грамматики G с правилами вывода $S \rightarrow ScS | b | a$
возможны следующие эквивалентные выводы терминальной цепочки
 acb : $S \rightarrow ScS \rightarrow Scb \rightarrow acb$
и $S \rightarrow ScS \rightarrow acS \rightarrow acb$.

Дерево вывода

- Деревом вывода цепочки x в КС-грамматике $G = (V_T, V_N, P, S)$ называется упорядоченное дерево, каждая вершина которого помечена символом из множества $V \cup V_N \cup \{\varepsilon\}$ так, что каждому правилу $A \rightarrow b_1 b_2 b_3 \dots b_k$, используемому при выводе цепочки x , в дереве вывода соответствует поддереву с корнем A и прямыми потомками $b_1, b_2, b_3, \dots, b_k$



Дерево вывода

- Дерево вывода часто называют деревом грамматического разбора, или синтаксическим деревом, а процесс построения дерева вывода - грамматическим разбором (синтаксическим анализом).
- КС-грамматика $G = (V_T, V_N, P, S)$ называется неоднозначной (неопределенной), если существует цепочка $x \in L(G)$, имеющая два или более дерева вывода.

Неоднозначные грамматики

Пусть даны две КС-грамматики:

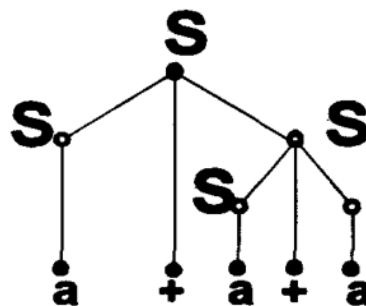
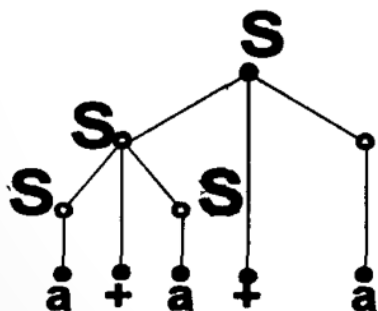
- $G_1 = (V_T, V_N, P_1, S), V_N = \{S\},$
- $P_1 = \{S \rightarrow S+S \mid S \mid S \cdot S \mid (S) \mid a\};$
- $G_2 = (V_T, V_N, P_2, S), V_N = \{S, A, B\},$
- $P_2 = \{S \rightarrow S + A \mid A, A \rightarrow A \cdot B \mid A / B \mid B, B \rightarrow a \mid (S)\},$

содержащие в множестве терминальных символов знаки операций, круглые скобки и символ a .
Определить, являются ли грамматики однозначными.

- Если какая-либо из них неоднозначна, привести пример цепочки, для которой существует два различных дерева вывода.

Неоднозначные грамматики

- Грамматика G_1 является неоднозначной, так как она имеет правила с правой частью, содержащей два вхождения нетерминала S и знак арифметической операции.
- Построим два дерева вывода для простейшей цепочки $a + a + a$:

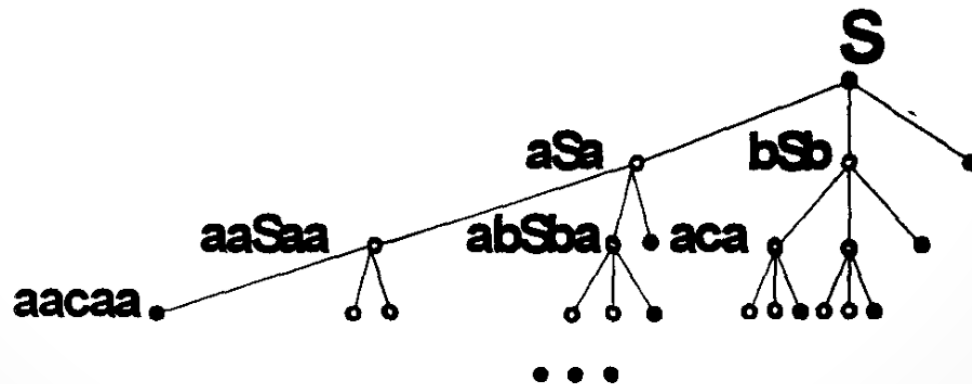


Грамматический разбор

- Различают две стратегии грамматического разбора: восходящую и нисходящую, которые соответствуют способу построения синтаксических деревьев.
- Преобразование цепочки, обратное порождению, называется редукцией.

Представление грамматики в виде графа

- Граф грамматики в качестве вершин содержит сентенциальные формы (любые цепочки, выводимые из аксиомы).
- Рассмотрим представление грамматики G в виде графа: $G = (V_T, V_N, P, S)$, в которой $V_T = \{a, b, c\}$, $V_N = \{S\}$, $P = \{S \rightarrow aSa \mid bSb \mid c\}$.



Преобразования КС-грамматик

- Часто требуется изменить грамматику таким образом, чтобы она удовлетворяла определенным требованиям, не изменяя при этом порождаемый грамматикой язык.
- Для этого используются эквивалентные преобразования КС-грамматик, некоторые из которых рассмотрены ниже.

Удаление правил вида

$A \rightarrow B$

- Преобразование первого типа состоит в удалении правил $A \rightarrow B$, или $\langle \text{нетерминал} \rangle \rightarrow \langle \text{нетерминал} \rangle$.
- Для любой КС-грамматики можно построить эквивалентную грамматику, не содержащую правил вида: $A \rightarrow B$, где A и B - нетерминальные символы.
- Пусть имеется КС-грамматика $G=(V_T, V_N, P, S)$, где множество нетерминалов $V_N = \{A_1, A_2, \dots, A_n\}$. Разобьем P на два непересекающихся множества: $P = P_1 \cup P_2$. В P_1 включены все правила вида $A_i \rightarrow A_k$, в P_2 включены все остальные правила, т.е. $P_2 = P \setminus P_1$.

Удаление правил вида

$A \rightarrow B$

- Пример:
- Пусть задана грамматика G со следующими правилами вывода
 $S \rightarrow aFb \mid A$; $A \rightarrow aA \mid B$; $B \rightarrow aSb \mid S$; $F \rightarrow bc \mid bFc$.
- Решение:
- Построим множества правил P_2 , $P(S)$, $P(A)$, $P(B)$, $P(F)$.

Удаление правил вида

$$A \rightarrow B$$

- Определим правила для P_2 : $P_2 = \{S \rightarrow aFb; A \rightarrow aA; B \rightarrow aSb; F \rightarrow bc \mid bFc\}$.
- Определим правила для $P(S)$: $S \Rightarrow A \Rightarrow B$ или $S \Rightarrow^* A$; $S \Rightarrow B$, где $\Rightarrow^*$ обозначает непосредственную выводимость. $P(S) = \{S \rightarrow aA; S \rightarrow aSb\}$.
- Определим правила для $P(A)$: $A \Rightarrow B \Rightarrow S$ или $A \Rightarrow^* B$; $A \Rightarrow S$. $P(A) = \{A \rightarrow aSb; A \rightarrow aFb\}$.
- Определим правила для $P(B)$: $B \Rightarrow S \Rightarrow A$ или $B \Rightarrow^* S$; $B \Rightarrow^* A$. $P(B) = \{B \rightarrow aFb; B \rightarrow aA\}$.
- Определим правила для $P(F)$: так как непосредственно выводимых нетерминалов не существует, то $P(F) = 0$.

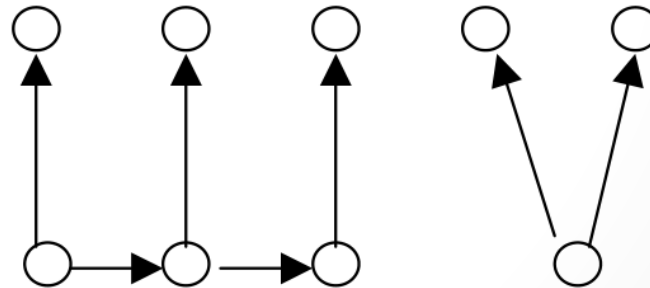
Удаление правил вида

$A \rightarrow B$

- Объединив полученные правила, можно записать грамматику G_3 , эквивалентную исходной:
- $S \rightarrow aFb \mid aSb \mid aA$; $A \rightarrow aA \mid aSb \mid aFb$;
- $B \rightarrow aA \mid aSb \mid aFb$; $F \rightarrow bc \mid bFc$.
- Исходная грамматика:
- $S \rightarrow aFb \mid A$; $A \rightarrow aA \mid B$; $B \rightarrow aSb \mid S$; $F \rightarrow bc \mid bFc$.

Графическая модификация метода

- При автоматизированном преобразовании грамматик проще применить графическую модификацию этого метода.
- С этой целью каждому нетерминальному символу и каждой правой части правил из множества P_2 поставлена в соответствие вершина графа.
- $S \rightarrow aFb \mid aSb \mid aA;$
- $A \rightarrow aA \mid aSb \mid aFb ;$
- $B \rightarrow aA \mid aSb \mid aFb;$
- $F \rightarrow bc \mid bFc.$



Удаление правил с терминальной правой частью

- Пусть в грамматике G имеется правило с терминальной правой частью $A \rightarrow v$, где $v \in V_T$. Тогда любой вывод с использованием этого правила имеет вид:
 $S \rightarrow^* xAy \rightarrow xv y$.
- Для того чтобы удалить терминальное правило грамматики $A \rightarrow v$, необходимо учесть следующие правила:
- а) если для нетерминала A больше нет правил, тогда во всех правых частях A заменяется на v ;
- б) если для нетерминала A есть другие правила, тогда добавляются новые правила, в которых A заменяется на v .

Удаление правил с терминальной правой частью

- Пример:
- Пусть дана КС-грамматика G :
 $S \rightarrow aSb \mid bAa \mid V$; $A \rightarrow ABa \mid b \mid \varepsilon$; $V \rightarrow ab \mid ba$.
- Удалим правила для нетерминала V . Тогда эквивалентная грамматика G_1 будет включать следующие правила:
- $S \rightarrow aSb \mid bAa \mid ab \mid ba$; $A \rightarrow Aaba \mid Abaa \mid b \mid \varepsilon$.

Задания для самостоятельной работы

1. Построить КС-грамматику, порождающую язык a^n для $n > 0$.
2. Построить КС-грамматику, порождающую язык $a^n b^n$ для $n > 0$.
3. Построить КС-грамматику, порождающую язык $(ab)^n c^* a^{n+m}$ для $n > 0$ и $m > 0$.
4. Построить КС-грамматику, порождающую язык $(ab)^* c a^* b^{n+m}$ для $n > 0$ и $m > 0$.
5. Построить КС-грамматику, порождающую язык $a^n b^{3m} c^m a^{2n} = a^n (bbb)^m c^m (aa)^n$ для $n > 1$ и $m > 1$.

- Построить КС-грамматику, порождающую язык $(ab)^n(ca)^*b^{n+2}(abc)^*$ для $n > 0$.